GISA
That's IT.

Teil I

Entwicklung verteilter Anwendungen

Anforderungen an heutige Applikationen – Monolith – Microservices – Cloud Native

---

Teil II

Containerisierung mit **Docker**

Virtualisierung – Container – Docker

Containerplattform mit **Kubernetes**

Orchestrierung - Cluster

Jonathan Stapf

30. Juni 2022

GISA
That's IT.

Teil I

Entwicklung verteilter Anwendungen

Anforderungen an heutige Applikationen – Monolith – Microservices – Cloud Native

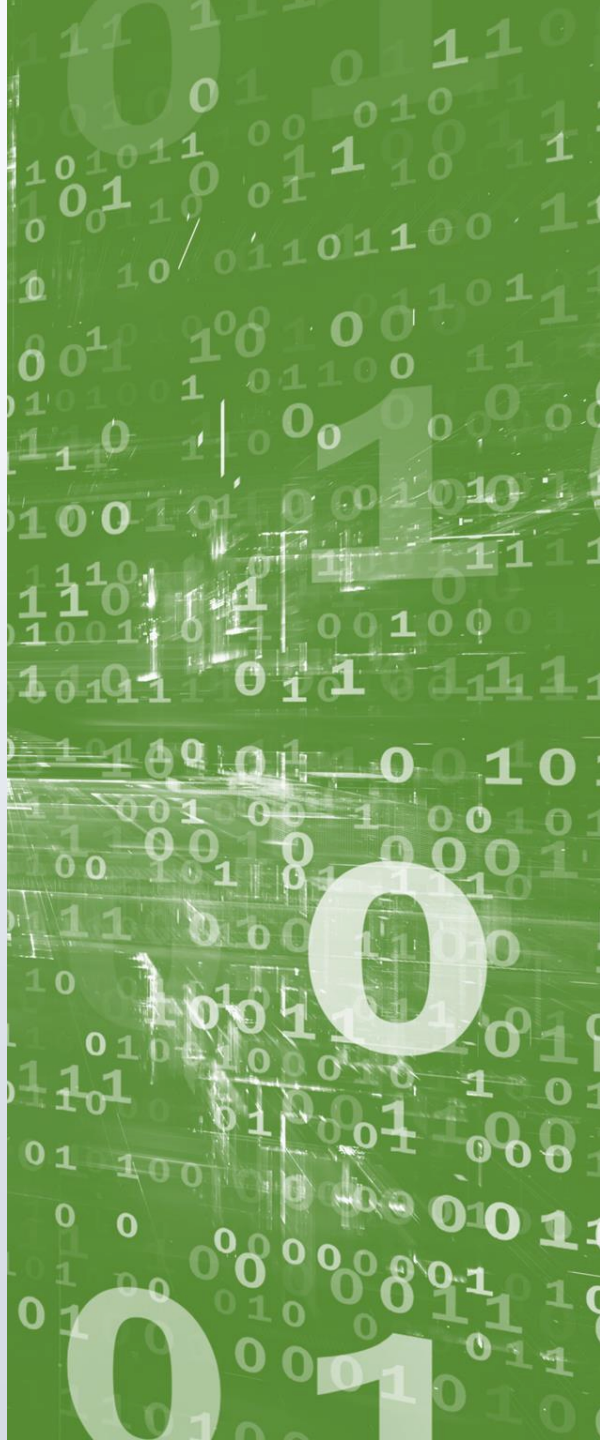
Jonathan Stapf

30. Juni 2022

Anforderungen an heutige Applikationen

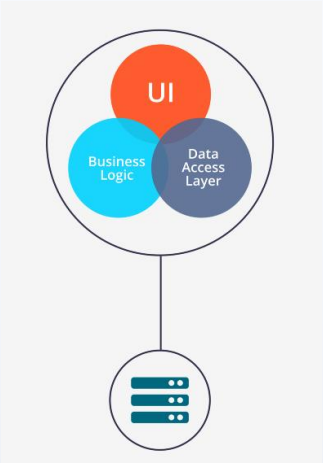
Vom Monolith zum Microservice

- **Zero Downtime**, always online – keine Ausfälle oder Wartungsfenster tolerierbar
- **Application Lifecycle**, Time-to-Market – kurze Entwicklungszyklen (Sprints), kleine Releases & Build-Artifacts
- Multidevice Support – ein Service kann von einer im Browser laufenden JavaScript-Applikation, einer nativen App unter Android/iOS oder gar M2M konsumiert werden
- **On-Demand Skalierbarkeit**, Ausfallsicherheit, Monitoring, Analytics
- Regelmäßige Updates – **Security & Bug Fixes** zum Schutz vor zufälligen oder gezielten Angriffen
- Weltweiter Zugriff – massiver Anstieg an Traffic und Datenvolumen als **Kostenfaktor**
- Georedundanz, mehrere Instanzen verteilt auf geografisch getrennte Data Center
- Trends / neue Paradigmen – Reactive Programming



Monolith – Pattern

Blick in die Vergangenheit - Never Touch a Running System



Quelle: https://devopedia.org/

Infrastruktur

- Mehrstufige Systemlandschaft – Development, Test und Production
- Vertikale Skalierung – mehr CPU, RAM und Storage
- Horizontale Skalierung – ein weiterer Applikationsserver mit identische Konfiguration (Hochverfügbarkeit auf Serverebene)
- Konfiguration der Systeme unterscheidet sich mit der Zeit
- Releasewechsel der Plattform oder Updates von gemeinsam genutzten Ressourcen mit nicht absehbaren Folgen
- Stabilität durch wenig Änderung - **„Change is the Exception“**

Applikationen

- gemeinsam nutzbare Ressourcen, Bibliotheken und Frameworks, wie z. B. Authentifizierung, Spring-Framework
- Deployment unter Berücksichtigung gegenseitiger Abhängigkeiten
- Applikationen sind anfällig für Ausfall der Infrastruktur
- Mehrere Teams entwickeln parallel – Absprachen und Abhängigkeiten erhöhen Aufwand für Entwicklung und Test
- Paralleles Deployment mehrere Instanzen einer Anwendung, z. B. für Test unterschiedlicher Features nicht möglich

Client Server Modell

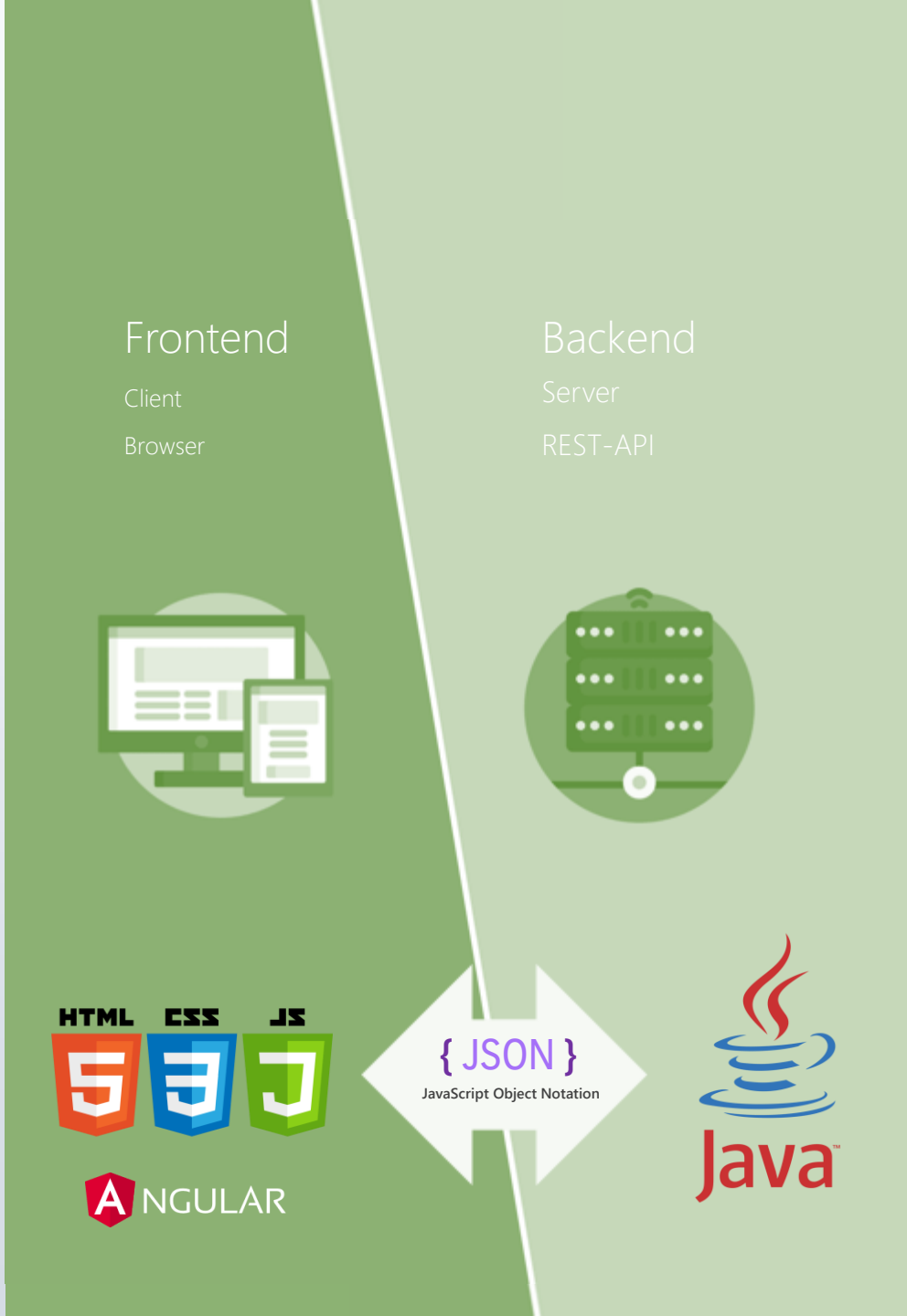
Auf dem Weg zu verteilten Anwendungen

⌛ Code bislang nur auf Server ausgeführt – serverseitig generiertem Content

JSP, HTML in PHP

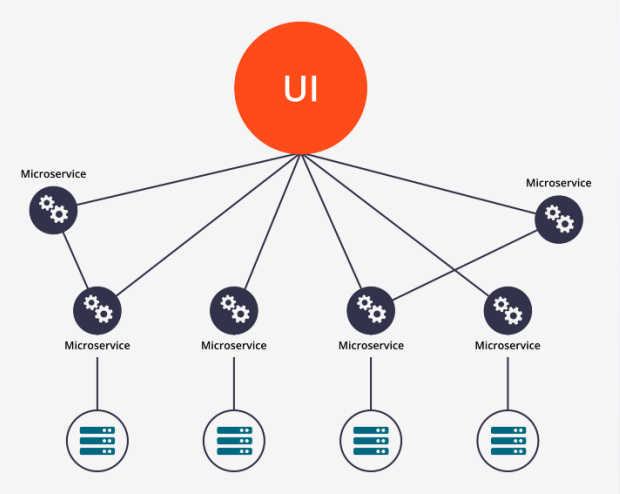
- **Entwicklung in getrennten Modulen** – Teile der Applikationen laufen im Browser (Frontend/UI) und Teile werden auf dem Server ausgeführt (Backend)
- verschiedenen Datenquellen (Datenbanken)
- **REST-API**: Kommunikation zwischen Client und Server über Netzwerk (JSON)

- Trennen und Reduzieren der Verantwortlichkeiten
- Teams können unabhängig voneinander an Teilen der Applikation entwickeln



Verteilte Anwendungen - **Microservices**

Seperation of Concerns



Quelle: https://devopedia.org/

- **Modulares Design** – kleine, verteilte Applikationen, die miteinander kommunizieren
- Seperation of Concerns – Services bilden nur einen Teil des Geschäftsprozess ab
- Deployment auf Hosts (Nodes), die nur die tatsächlich benötigten Ressourcen bereitstellen
- Kommunikation über Netzwerk (Internet), Request/Response und Event-Driven

Vorteile

- Teams können unabhängig voneinander, im eigenen Stack, entwickeln
- Fällt ein Teil ein Service aus, funktionieren die anderen Services weiterhin, z. B wird die Abarbeitung von Bestellaufträgen ausgesetzt (Queue) und dann fortgesetzt, wenn Service wieder verfügbar ist
- kleine lokale Datenbanken, meist nur Nutzdaten des Services selbst

Die Komplexität steigt

- Schnittstellen können sich ändern
- Authentifizierung, Stateless vs. Stateful
- Toleranz gegenüber Latenzen oder Ausfälle von Netzwerk
- Zusammenspiel der Services ist schwer zu testen
- CAP Theorem

Cloud Native – Architectural Pattern

Cloud is about where the Application is hosted. Cloud Native is how.

- Cloud Native beschreibt **Design Pattern** für Applikationen und wie diese Applikationen deployed werden
- **Cloud Native Applications** - Distributed Systems / Verteilte Anwendungen / Microservices
- Applikationen werden in **Containern** deployed
- **Kubernetes** - die Plattform für Cloud Native Applikationen

Cloud Native Applications - Software, die modular aufgebaut, skalierbar, anpassungsfähig und unempfindlich auf Infrastrukturausfälle reagiert, die in sich permanent ändernder Umgebung / Infrastruktur betrieben werden kann und selbst kontinuierlichen Veränderungen ausgesetzt ist.

CLOUD NATIVE Landscape

KubeCon | CloudNativeCon

Cloud Native Applications

Cloud Native Data

Cloud Native Interactions

Continious Delivery

Release Frequently

Change is the Rule

Safe Deployments

Highly Automated

Repeatability

Resilient

Infrastructure Changes

Die Rolle des **Entwicklers**

Deployment Failed in Production. „But it works on my Machine!“

„Warum geht mich das als Entwickler etwas an?“

- Firefighting vs. Release Features – Creating Value for the Product
- Abweichende Konfiguration und Deployments in verschiedenen Stages
- eigenes Build-Artifakt für jede Stage: jdbc-config, application.properties
- Snowflakes, Risky Deployments, Technical Depth, Set-Up-Time

- ✓ **Desired State** - Zustand der Software ist definiert und versioniert für wiederholbares Deployment: JSON, YAML
- ✓ Kurze Entwicklungszyklen
- ✓ **Pattern, Technologie, Security** – lebenslanges Lernen

Software Developer FullStack Developer

Cloud Infrastructure Engineer DevOps Engineer

Frontend | Backend Developer Site Reliability Engineer

Database Engineer Security Engineer

Scrum Product Owner Software Architect

GISA
That's IT.

Teil II

Containerisierung mit **Docker**

Virtualisierung – Container – Docker

Containerplattform mit **Kubernetes**

Orchestrierung - Cluster

Jonathan Stapf

30. Juni 2022

Virtualisierung mit **Containern**

Grundlagen Container Technologie

Idee

- **Seperation of Concerns** – Modularisierung und Trennen nach Aufgaben
- On-Demand Erweitern oder Verringern der Ressourcen
- Software leichter auf unterschiedlichen Systemen ausrollen z. B. auf Test-System, Production-System, Sandbox-System (...)
- Flexibilität und kontinuierliche Veränderungen erlauben

Der Weg zur Lösung

- Applikationen geeignet paketieren
- Deployment und Konfiguration der Applikationen vereinheitlichen
- Geeignete Infrastruktur bereitstellen
- **Wiederholbare Schritte** definieren
- Abhängigkeiten minimieren

Virtualisierung mit **Containern**

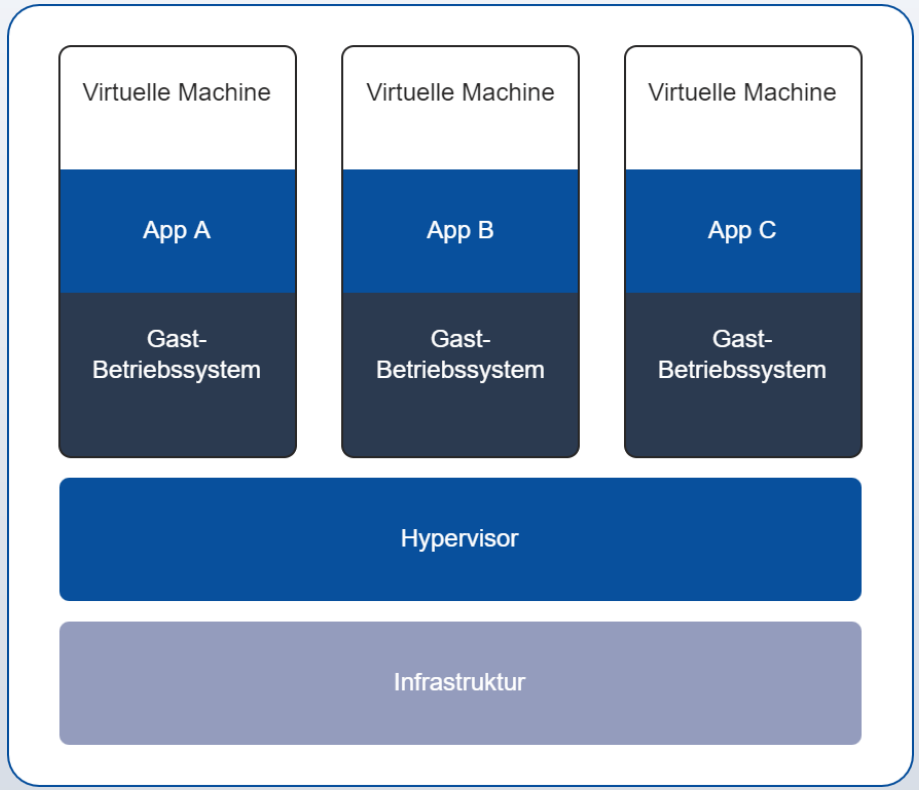
Grundlagen Container Technologie

Container

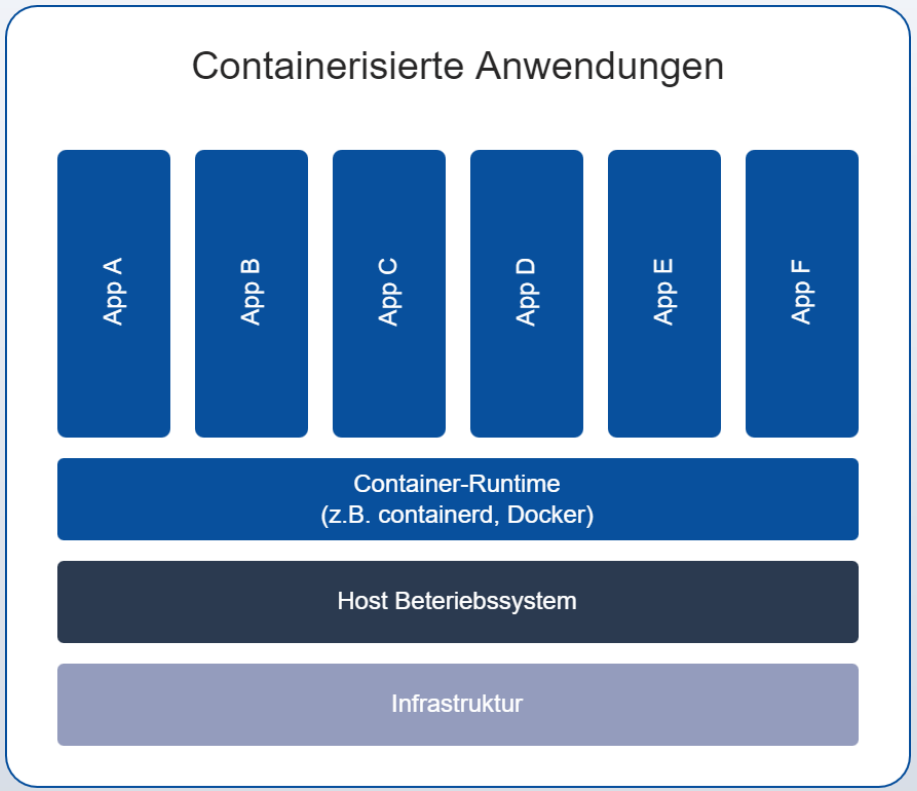
- Enthalten, neben der Applikation, nur die **notwendigen Abhängigkeiten** mit einem **Minimum an Betriebssystem**
- lassen sich auf dem Hostsystem starten, stoppen, austauschen, löschen (...)
- Haben keine Abhängigkeiten zu anderen Prozessen, aber **nutzen gemeinsam** den **Kernel des Hostsystems**
- Ein Prozess pro Container
- **Portabilität** - immergleiche Installation auf unterschiedlichen Hosts (Nodes) mit Container-Runtime
- **Isolation** - Trennung der benutzten Systemressourcen (Prozesse, Dateisystem, Netzwerkschnittstellen)
- **Parametrisierung** – Konfiguration über Environment-Variablen

Virtualisierung mit **Containern**

Grundlagen Container Technologie



VS



Nachgezeichnet von Docker.com (https://www.docker.com/resources/what-container)

Containerisierung mit **Docker**

Container-Runtime, Container-Engine

Container-Runtime

- Software, die Container auf einem Host ausführt
- Automatisiert die Befehle für den Linux-Kernel zum Starten der isolierten Prozesse, die als Container bezeichnet werden

Container-Engine

- empfängt Befehle über seine API und ruft die Container-Runtime auf (Docker CLI, Docker Compose, Kubelet)
- Managed den Container-Lifecycle

Docker

- Ist eine **Container-Engine** mit containerd (runC, OCI) als Runtime
- Hat die Containertechnologie populäre gemacht
- Build, Ship & Run – vereinfacht das Verpacken von Applikationen und deren Konfiguration in Form von **Images** und das Deployen von **Containern**
- Alternativen
  - containerd mit CTR
  - CRI-O mit CRICTL
  - Podman („deamonless“)

Containerisierung mit **Docker**

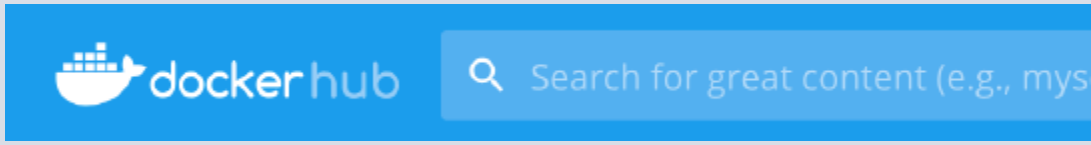
Images - Container at Rest

Image

- Paket mit der Applikation und den Abhängigkeiten, in der Regel eine oder mehrere Dateien
- bestehen aus Layern (Schichten) und MetaData
- In den Layern werden die Applikationen, benötigten Bibliotheken, Daten, Systemabhängigkeiten gespeichert
- Verwaltung in einem Container-Repository (Registry, Docker-Hub)
- Versionierung mit Tags
- Standard für Image-Formate – Open Container Initiative (OCI)

Dockerfile

- Dockerfile definiert die Bestandteile eines Images (Basisbetriebssystem, Anwendungen, Ports usw.)
- Jeder Befehl legt erzeugt einen neuen Layer
- Blueprint, Build-Manifest für ein Image

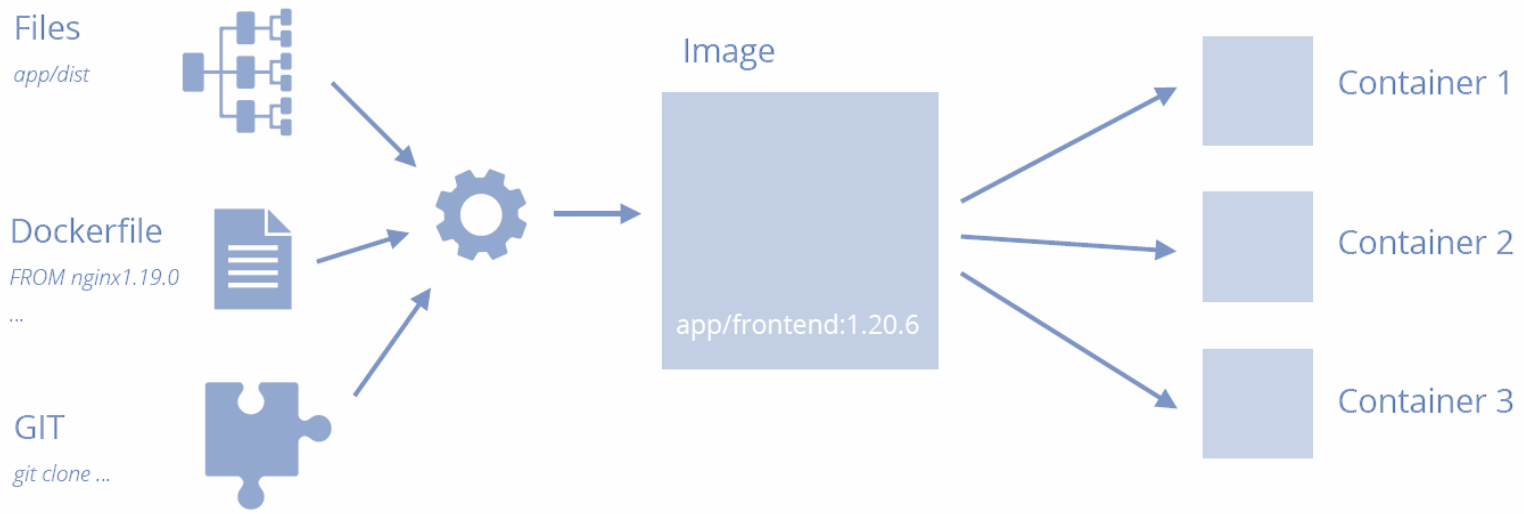


Containerisierung mit **Docker**

Images, Container

Container

- Container sind laufende Instanzen auf Basis eines Images
- jeder laufende Container ist ein isolierter Prozess auf dem Host
- Container und das Host-Betriebssystem greifen auf den selben laufenden Linux-Kernel zurück



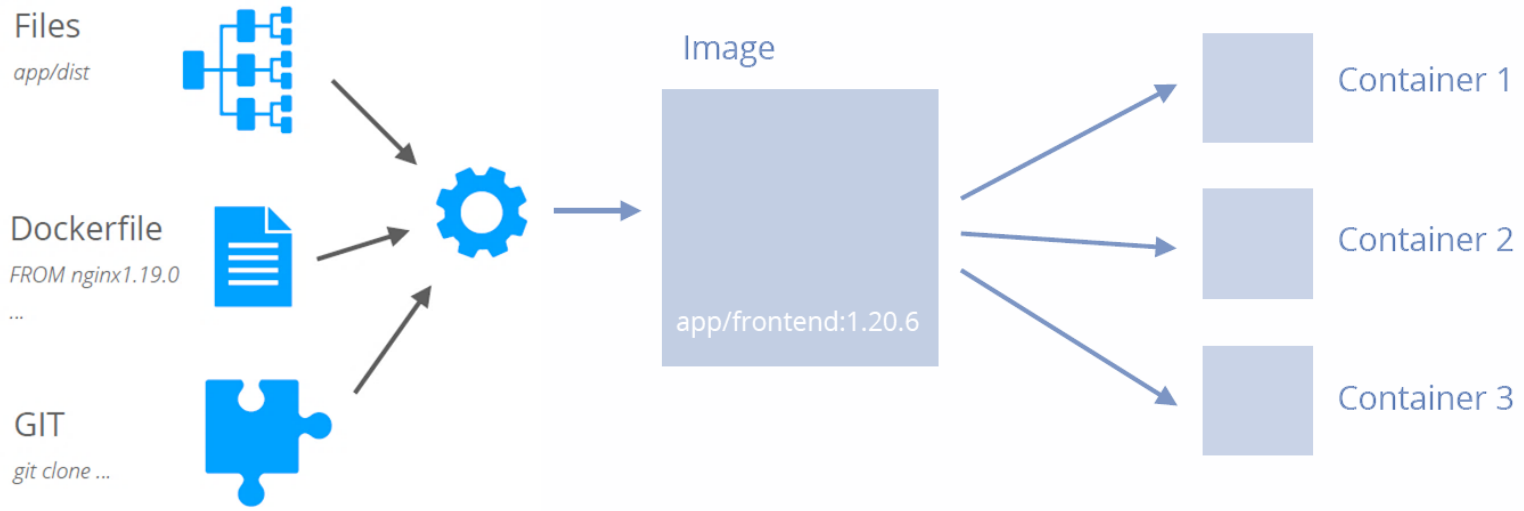
- es können beliebig viele Container von einem Image gestartet werden
- viele Container laufen parallel auf dem gleichen Host
- kurzlebig und jederzeit reproduzierbar

Containerisierung mit **Docker**

Images, Container

Container

- Container sind laufende Instanzen auf Basis eines Images
- jeder laufende Container ist ein isolierter Prozess auf dem Host
- Container und das Host-Betriebssystem greifen auf den selben laufenden Linux-Kernel zurück



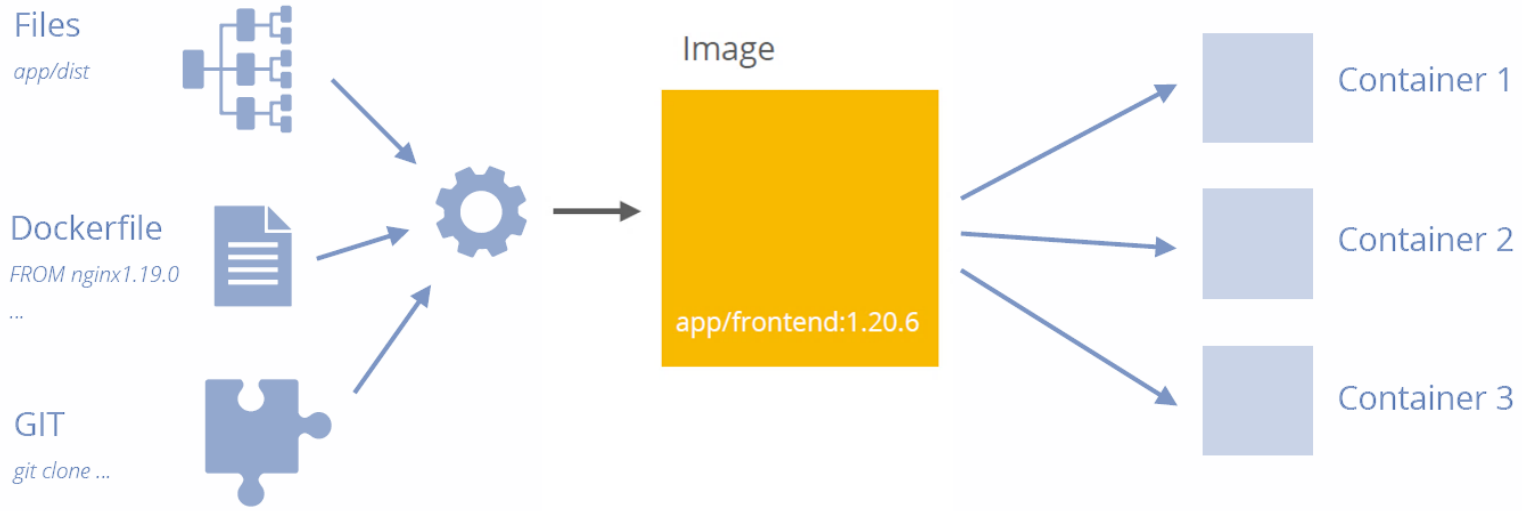
- es können beliebig viele Container von einem Image gestartet werden
- viele Container laufen parallel auf dem gleichen Host
- kurzlebig und jederzeit reproduzierbar

Containerisierung mit **Docker**

Images, Container

Container

- Container sind laufende Instanzen auf Basis eines Images
- jeder laufende Container ist ein isolierter Prozess auf dem Host
- Container und das Host-Betriebssystem greifen auf den selben laufenden Linux-Kernel zurück



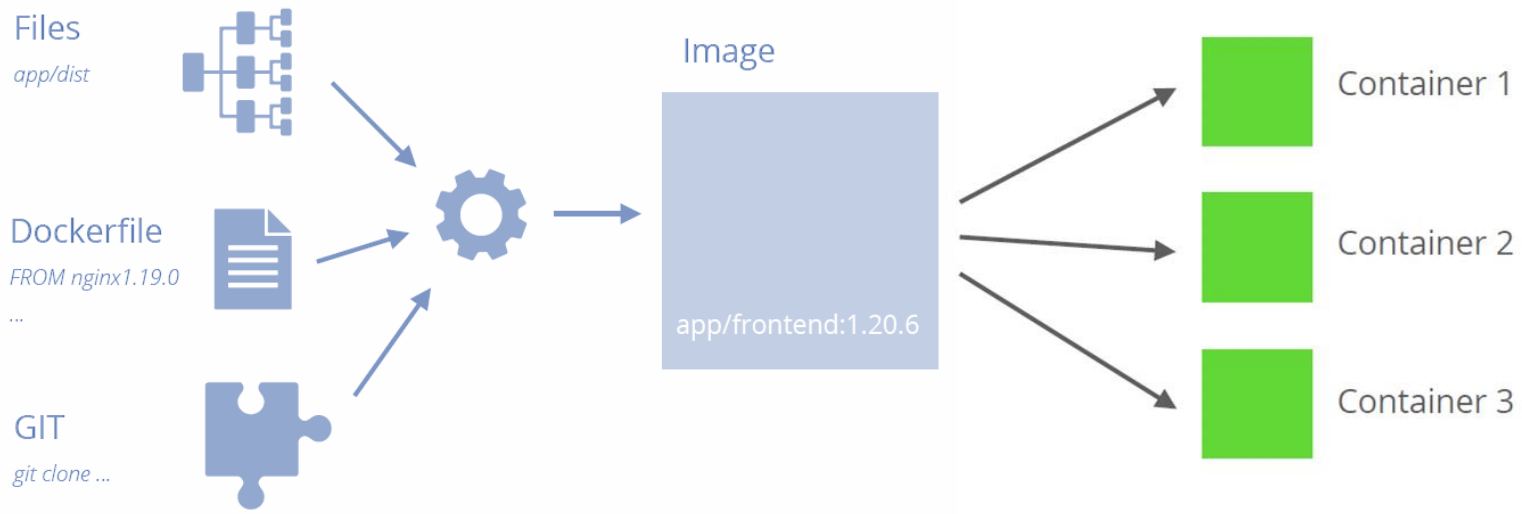
- es können beliebig viele Container von einem Image gestartet werden
- viele Container laufen parallel auf dem gleichen Host
- kurzlebig und jederzeit reproduzierbar

Containerisierung mit **Docker**

Images, Container

Container

- Container sind laufende Instanzen auf Basis eines Images
- jeder laufende Container ist ein isolierter Prozess auf dem Host
- Container und das Host-Betriebssystem greifen auf den selben laufenden Linux-Kernel zurück



- es können beliebig viele Container von einem Image gestartet werden
- viele Container laufen parallel auf dem gleichen Host
- kurzlebig und jederzeit reproduzierbar

Registry

- Speichern und Veröffentlichen von Images - **push**
- Download von fertigen Images - **pull**
- Security-Scans
- Images können auf verschiedenen Hosts verteilt werden
- Docker Hub, Google k8s.gcr.io, Microsoft, ...

Private Registry

- zentrales Speichern von Images innerhalb eines Unternehmens
- Zugangskontrolle / welche Images dürfen verwendet werden
- „Proxy“ für externe Registries
- „air-gapped“ Umgebungen
- Signieren von Images
- CNCF Harbor, Nexus, Azure CR, ...

Docker Hub

- bekannteste und größte **Public Registry**
- betrieben durch Docker Inc.
- fertige Container-Images
- durch Hersteller oder Dritte bereitgestellt
- vertrauenswürdig vs. nicht vertrauenswürdig

Link: *https://hub.docker.com/*

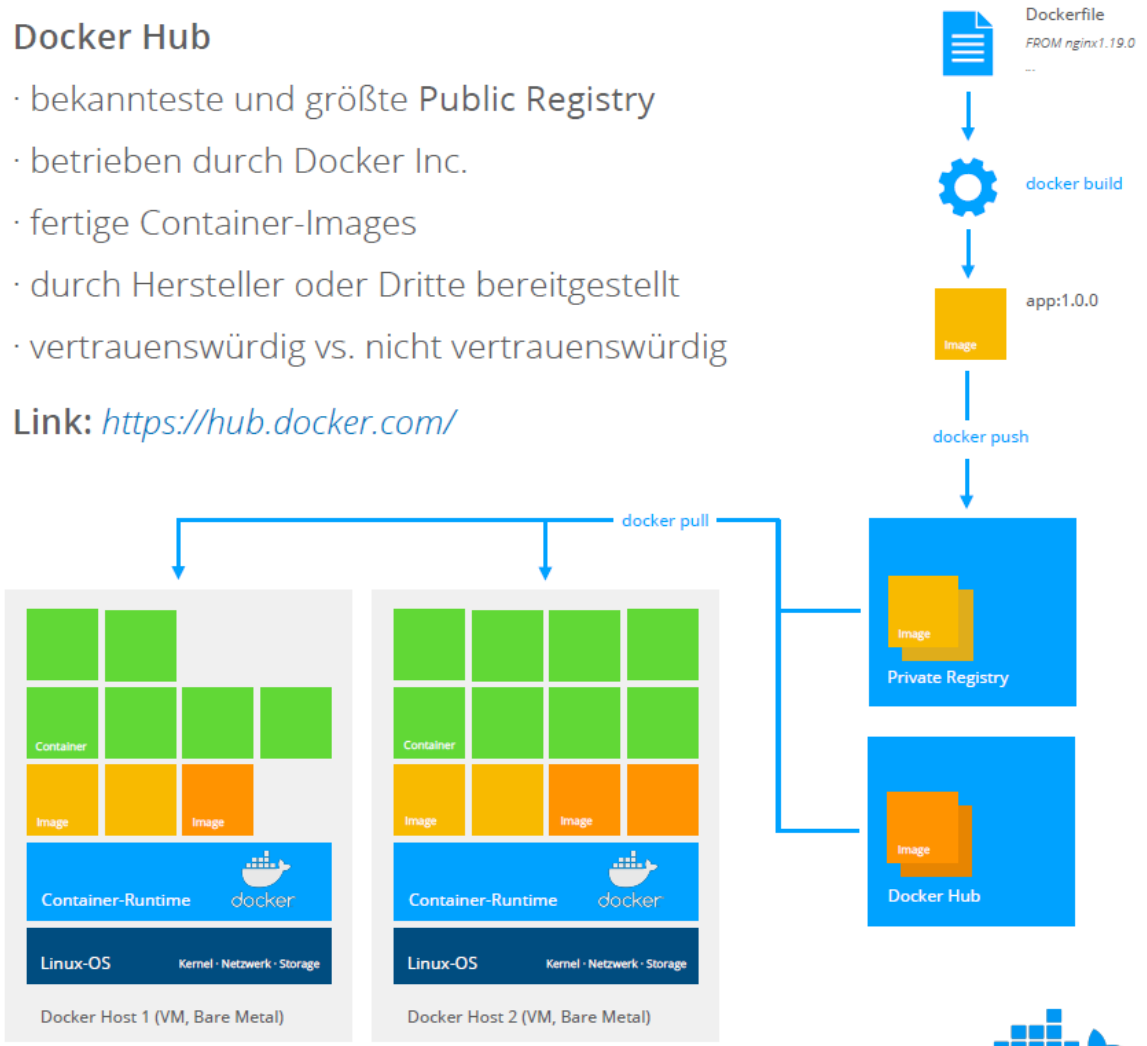


Abb.: Die Schritte eines Images vom Build zur Nutzung auf einem Docker Host

Dockerfile

Befehle

FROM: Base-Image · kann ein beliebiges Image sein

ENV: definiert eine Environment-Variable im Image

- steht im Build-Prozess zur Verfügung
- steht im Container zur Laufzeit zur Verfügung

RUN: Führt einen Shell-Befehl aus, z. B.:

`· apt-get install · git clone · npm install`

COPY: kopiert Files relativ zum Dockerfile in das Image

```
copy [Pfad auf Host] [Pfad im Container]
```

EXPOSE: Port(s), über den Services im Container erreichbar sind

ENTRYPOINT: Start-Command für den Container

CMD: Default-Start-Command für den Container

Docs: https://docs.docker.com/engine/reference/builder/

https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

Dockerfile

```
FROM ubuntu:20.04

LABEL maintainer="Maintainer <vorname.nachname@gisa.de>"
# environment vars
ENV CMS_VERSION   5.1.3
ENV http_proxy   http://http-proxy:8080
# single line command
RUN addgroup -g 101 -S myGroup
# multi line command
RUN apt-get update && apt-get install -y \
    package-1 \
    package-2 \
    package-3
RUN make /somefolder/somefile
# copy entrypoint script
COPY docker-entrypoint.sh /
# entrypoint script
ENTRYPOINT ["/docker-entrypoint.sh"]
# default ports
EXPOSE 80
EXPOSE 443/tcp
EXPOSE 3000/udp
# default startup, run after entrypoint
CMD ["nginx", "-g"]
```

Layer eines „Docker“ Images

```
1  FROM nginx:1.18.1
2  LABEL maintainer="Hannes Hellmich <hannes.hellmich@gisa.de>"
3  ENV APP_VERSION 2.1.0
4  RUN apt-get update && apt-get install -y \
5      package-x \
6      package-y \
7      package-z
8  COPY ./app /usr/share/nginx/html
```

jeder Befehl im Dockerfile erzeugt einen neuen Layer

Zeile 1: Base-Image

Zeile 2: Label / Maintainer

Zeile 4: Zusammenfassen mehrerer Befehle in einem Command

Zeile 5: Kopieren von Inhalten in das Image

letzter Layer ist „Arbeitsebene“ des Containers

Layer werden wiederverwendet

docker

GISA
That's IT.

Beispiel I

Docker Container

Docker Build – Docker Run

Jonathan Stapf

30. Juni 2022

Begriffe

Container · Image · Tag

Registry · Docker Hub · Private Registry

Images

Dockerfile · Layer · Erstellen · Pushen

Pullen · Versionierung · Tags · Security

Container

Starten · Daemon -d · Ports · ID & Name

Volumes & Mount

Netzwerke

DNS & IP

Kommunikation zwischen Containern

Kommunikation von und nach Außen

Ressourcen

Limits für CPU & RAM

Storage

Tools

CLI · Docker Compose · Portainer

Day Two Ops

Restarts · Health Checks · Monitoring

Logs · Debug

Best Practices

KISS & Single-Responsibility Principle

:latest Tags

Container-Lifecycle · Image & Stages

NGiNX-Container

docker

Orchestrierung von Containern

Vom Single-Node zur Containerplattform

Workloads

- Applikationen bestehen i.d.R. aus mehreren Containern
- die **Anzahl wächst** schnell, die Komplexität steigt
- es ist sinnvoller, die Workload auf **mehrere kleinere Hosts** zu verteilen (mit ggf. geringer Verfügbarkeit)
- die optimale Auslastung finden, um damit Kosten zu sparen
- Wunsch nach **Hochverfügbarkeit**
- Wunsch nach **Automatisierung** der Bereitstellung
- Wunsch nach Integration in den Entwicklungsworkflow

Containerplattform

- Betrieb der Containerplattform mit mehreren **Hosts** (Nodes), die **zu einem Cluster zusammengefasst** sind
- das Cluster wird **zentral gemanagt**
- Loadbalancing, Skalierung und Scheduling der Workloads in Form von Containern
- Kubernetes, Docker Swarm Mode, Apache Mesos, ...
- **Managed Kubernetes Services**: OpenShift, GKE, AKS, GISA

Kubernetes – Überblick

Plattform für Cloud Native Applikationen

- Open-Source-Plattform
- ursprünglich von Google entwickelt, 2015 der **Cloud Native Computing Foundation (CNCF)** gestiftet
- übernimmt die Orchestrierungs- und Managementfunktionen für eine Containerplattform
- ein Cluster kann jeweils aus mehreren Public-, Private- oder Hybrid-Cloud-Hosts (Nodes) bestehen
- steuert Container-Runtime, z. B. Docker
- Regeln legen fest, wie und wie viele Container auf die Nodes verteilt werden
- stellt sicher, dass immer die definierte Anzahl an Containern ausgeführt wird
- Deployment und (in-time) Skalierung der Applikationen
- ermöglicht automatische, lastabhängige Skalierbarkeit
- Ressourcen werden durch rollenbasierter Zugriff verwaltet
- wird ergänzt durch Tools für Networking, Storage, Monitoring, Security
- Integration in Entwicklungs-Pipelines

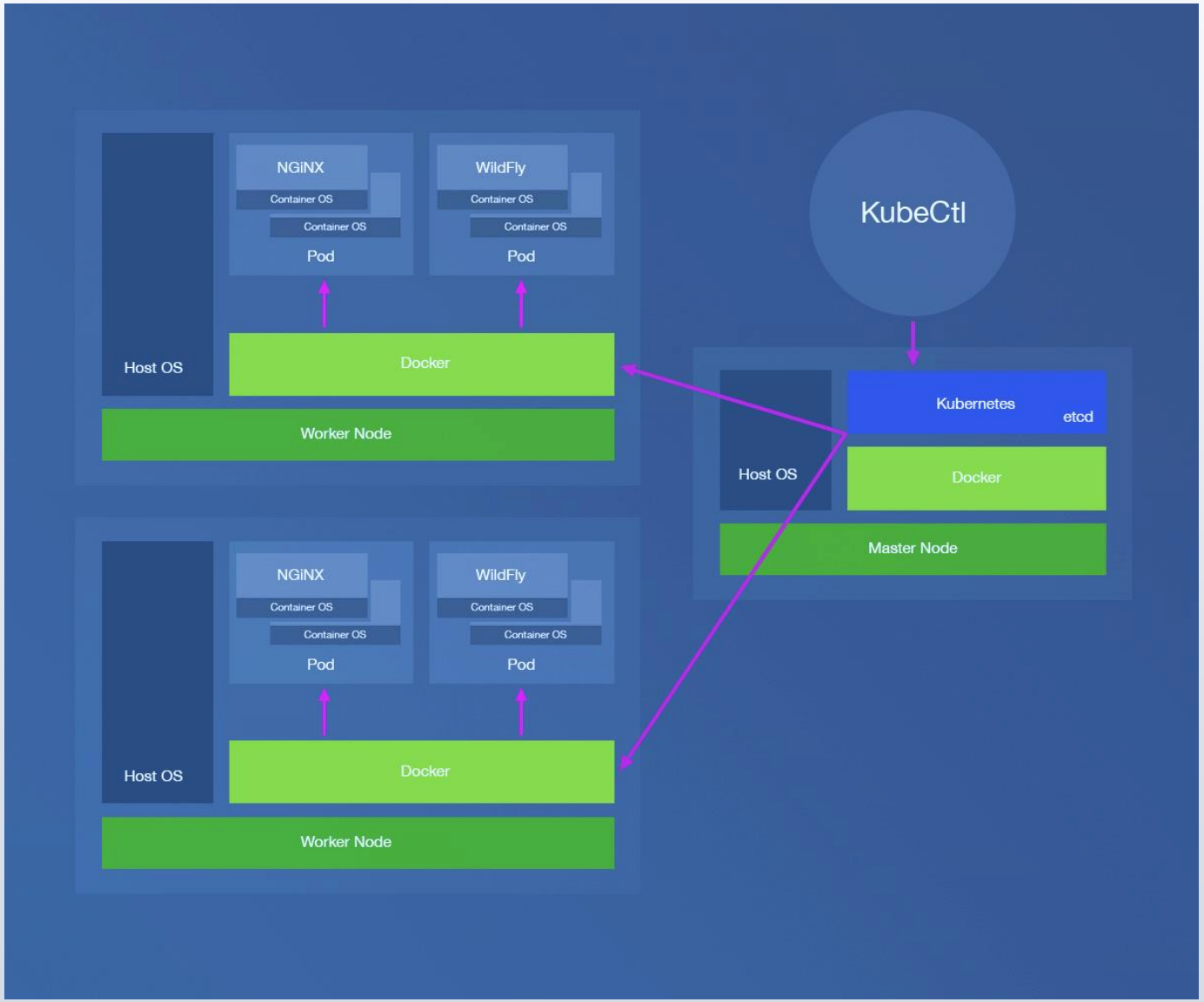
Kubernetes **Cluster**

Komponenten des Kubernetes Clusters

- **Master-Node** – ControlPlane, Verwaltung und Verteilung der Container
- KubeAPIServer – HTTP-Schnittstelle für Steuerbefehle zu Management der der Workloads / Container Lifecycle
- Etcd – speichert Konfigurations- und Laufzeitdaten
- KubeControllerManager, KubeScheduler

- **Worker-Node** – Host (Server), auf denen die Workloads (Container) ausgeführt werden
- Kubelet steuert die Container-Runtime

- **KubeCtl** – Commandline-Tool zum Interagieren mit KubeAPI



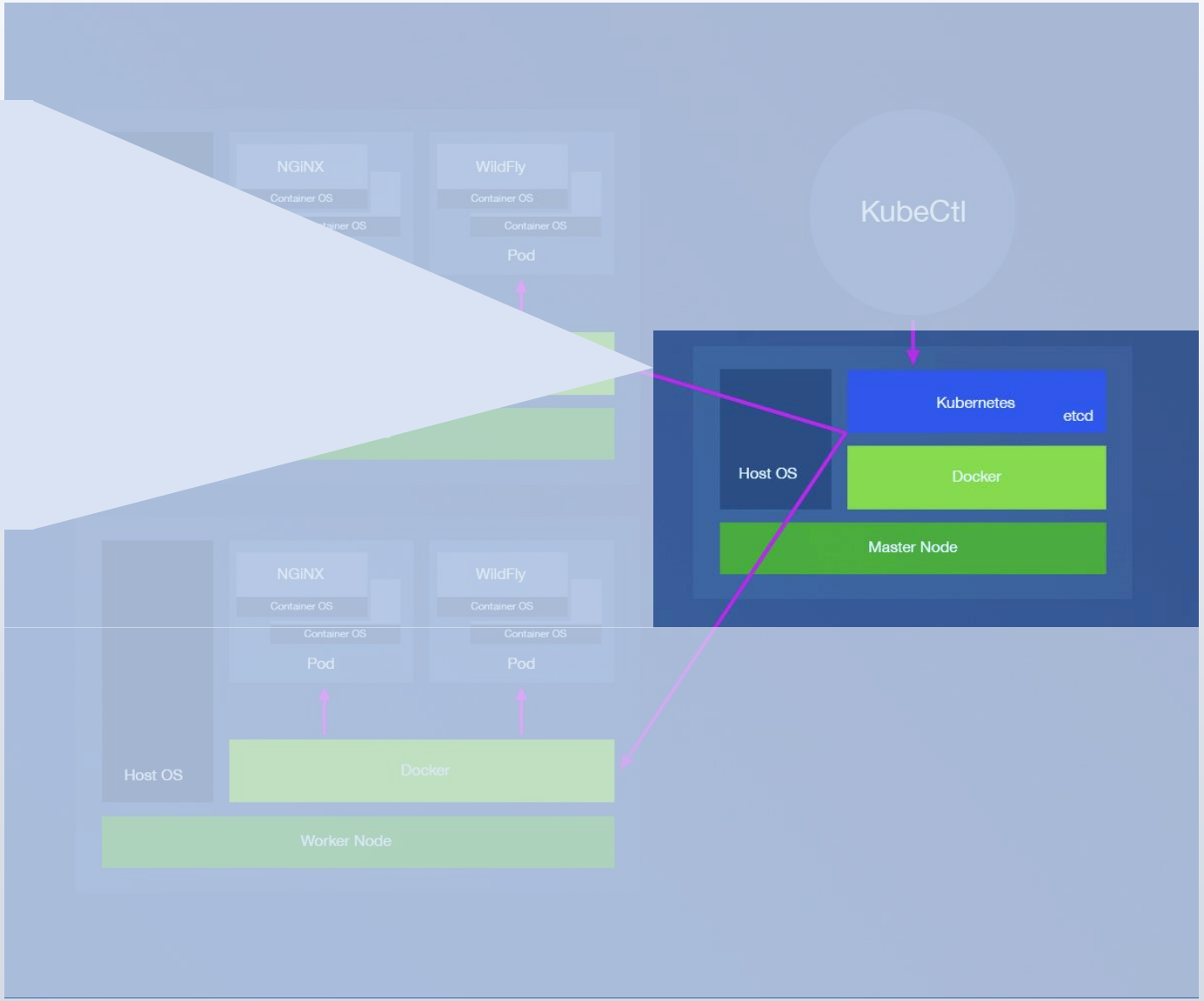
Kubernetes **Cluster**

Komponenten des Kubernetes Clusters

- **Master-Node** – Host (Server), ControlPlane, Verwaltung und Verteilung der Container
- KubeAPIServer – HTTP-Schnittstelle für Steuerbefehle zu Management der der Workloads / Container Lifecycle
- Etcd – speichert Konfigurations- und Laufzeitdaten
- KubeControllerManager, KubeScheduler

- **Worker-Node** – Host (Server), auf denen die Workloads (Container) ausgeführt werden
- Kubelet steuert die Container-Runtime

- **KubeCtl** – Commandline-Tool zum Interagieren mit KubeAPI



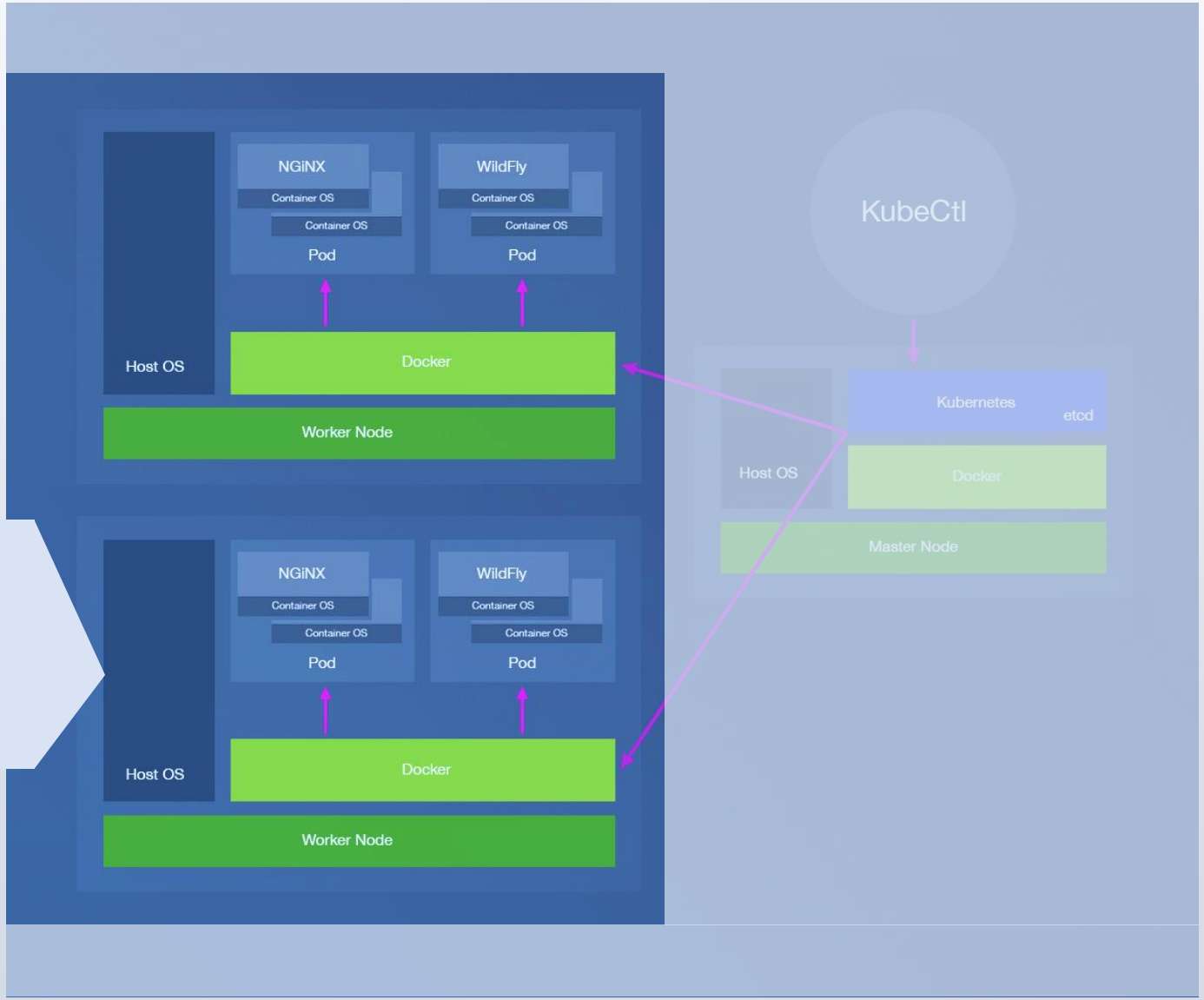
Kubernetes **Cluster**

Komponenten des Kubernetes Clusters

- **Master-Node** – Host (Server), ControlPlane, Verwaltung und Verteilung der Container
- KubeAPIServer – HTTP-Schnittstelle für Steuerbefehle zu Management der der Workloads / Container Lifecycle
- Etcd – speichert Konfigurations- und Laufzeitdaten
- KubeControllerManager, KubeScheduler

- **Worker-Node(s)** – Host (Server), auf denen die Workloads (Container) ausgeführt werden
- Kubelet steuert die Container-Runtime

- **KubeCtl** – Commandline-Tool zum Interagieren mit KubeAPI



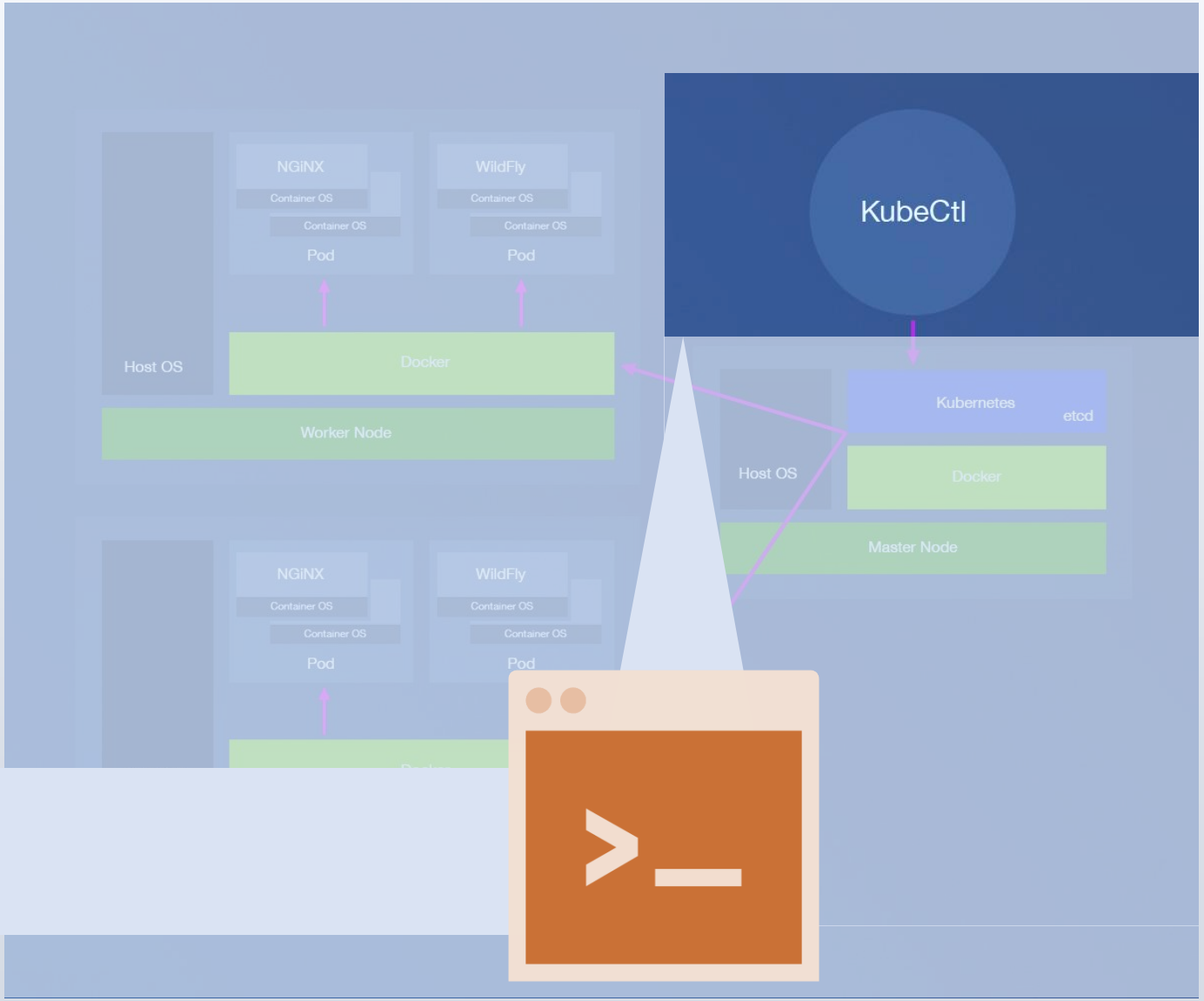
Kubernetes **Cluster**

Komponenten des Kubernetes Clusters

- **Controlplane-Node(s) (Master)** – Host (Server), ControlPlane, Verwaltung und Verteilung der Container
- KubeAPIServer – HTTP-Schnittstelle für Steuerbefehle zu Management der der Workloads / Container Lifecycle
- Etcd – speichert Konfigurations- und Laufzeitdaten
- KubeControllerManager, KubeScheduler

- **Worker-Node(s)** – Host (Server), auf denen die Workloads (Container) ausgeführt werden
- Kubelet steuert die Container-Runtime

- **KubeCtl** – Commandline-Tool zum Interagieren mit KubeAPI



Kubernetes – Terminologie

Wie werden Applikationen bereitgestellt?

Kubernetes Objects (Konfigurationsobjekte)

Um Workloads im Kubernetes Cluster zu deployen, müssen diese zunächst definiert werden.

- Dienen zur **Beschreibung des gewünschten Zustands** (State) einer Applikation im Cluster
- **CRUD** mit KubeCtl und Kubernetes API
- Einmal erstellt wird Kubernetes sicherstellen, dass dieses Objekt existiert
- Helm als `Pakete-Manger'

- **Namespaces** – Gruppierung von Workloads
- **Pod** – kleinste deploybare Einheit, definiert Container
- **Deployment** – Wrapper für Workloads, definiert Anzahl der der Pods (Replicas), Update-Strategie, Metadata
- **Service** – Loadbalancing
- **Ingress** – Entrypoint, Proxy
- RBAC
- StatefulSet, DaemonSet, ReplicaSet, ConfigMap, Secret
- **CRD** (Custom Resource Definition)

Kubernetes – Inside

Deklarative Beschreibung – Infrastructure as Code

YAML-Files (JSON-Files)

- Alle Kubernetes-Objekte werden in YAML-Files oder JSON-Files definiert
- vollständige Definition der Applikation
- versionierbar und archivierbar, z. B. mit GIT
- Mit **KubeCtl** werden die Operationen ausgeführt

```
kubectl -n my-namespace apply -f deployment.yaml
```

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: keycloak-app
  labels:
    app: keycloak
spec:
  replicas: 1
  selector:
    matchLabels:
      app: keycloak-app
      tier: backend
  template:
    metadata:
      labels:
        app: keycloak-app
        tier: backend
    spec:
      containers:
      - name: keycloak-application
        image: container-registry.intern/jboss/keycloak:13.0.0
        env:
        - name: PROXY_ADDRESS_FORWARDING
          value: "true"
        - name: DB_ADDR
          value: postgres-service
        ports:
        - name: http
          containerPort: 8080
        - name: https
          containerPort: 8443
        readinessProbe:
          httpGet:
            path: /auth/realms/master
            port: 8080
```

GISA
That's IT.

Beispiel II

Kubernetes Deployment

Demo – Rolling Upgrade

Jonathan Stapf

30. Juni 2022

Kubernetes Takeaways

Zusammenfassung

Container können…

- einheitlich auf unterschiedlichen Hosts (Nodes) mit einer Container-Runtime gestartet werden – **Portabilität**
- schnell auf mehreren Hosts deployed werden – **Geschwindigkeit**
- in unterschiedlichen Versionen **parallel auf einem Host** gestartet werden (Testsystem, Updates, Wiederherstellung)
- sämtliche Bausteine einer Applikation bilden
- Images werden in einer Registry, z. B. Docker Hub, verwaltet

Kubernetes …

- ist Orchestrierung von Containern
- **steuert** Container-Runtimes, nicht nur Docker
- High-Availability auf Anwendungsebene
- horizontale Skalierung für Applikationen
- nicht jede Applikation kann skalieren
- **automatisiert** Deployments, Updates, Rollbacks
- komplizierter Einstieg
- **Cloud Native**, Industriestandard

Links

Cloud Native

https://www.cncf.io/

Kubernetes

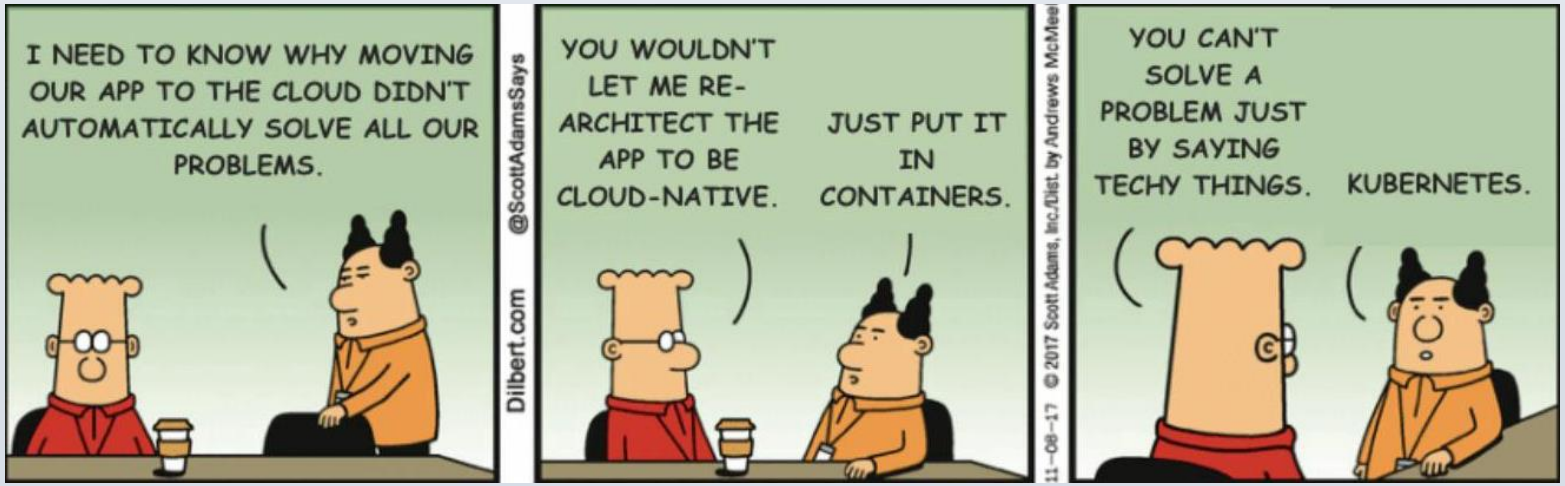
YouTube Channel // TechWorld with Nana

https://www.youtube.com/watch?v=X48VuDVv0do

Kubernetes @ Edge (Raspberry)

https://k3s.io/

Microservices

https://microservices.io/

https://medium.com/nerd-for-tech/microservice-architecture-622e4148f1